

FAST FOURIER TRANSFORM

DFT: $X[k] = \sum_{n=0}^{N-1} x[n] \cdot e^{-j \frac{2\pi}{N} k \cdot n}$, $k=0, 1, \dots, N-1$ where N is complex multiplies } for each $\underline{k}$
 when $N-1$ is complex adds

$\left[\begin{array}{l} O(N^2) \text{ computation for DFT} \\ \text{VS} \\ O(N \log_2 N) \text{ for FFT} \end{array} \right]$

FFT: exploits symmetries of $e^{-j \frac{2\pi}{N} k \cdot n}$ (the complex exp prop of DFT)
 Define $W_N = e^{-j \frac{2\pi}{N}}$

Separate $x[n]$ into even & odd-indexed sub sequences

$$X[k] = \sum_{n=0}^{N-1} x[n] W_N^{kn} = \sum_{\text{odd}} x[n] W_N^{kn} + \sum_{\text{even}} x[n] W_N^{kn}$$

even indices } $n=2r$, $r=0, 1, \dots, \frac{N}{2}-1$
 odd indices } $n=2r+1$

$$= \sum_{r=0}^{\frac{N}{2}-1} x[2r] W_N^{k \cdot 2r} + \sum_{r=0}^{\frac{N}{2}-1} x[2r+1] W_N^{2rk+k} \quad (\text{subst. } n=2r \text{ or } n=2r+1)$$

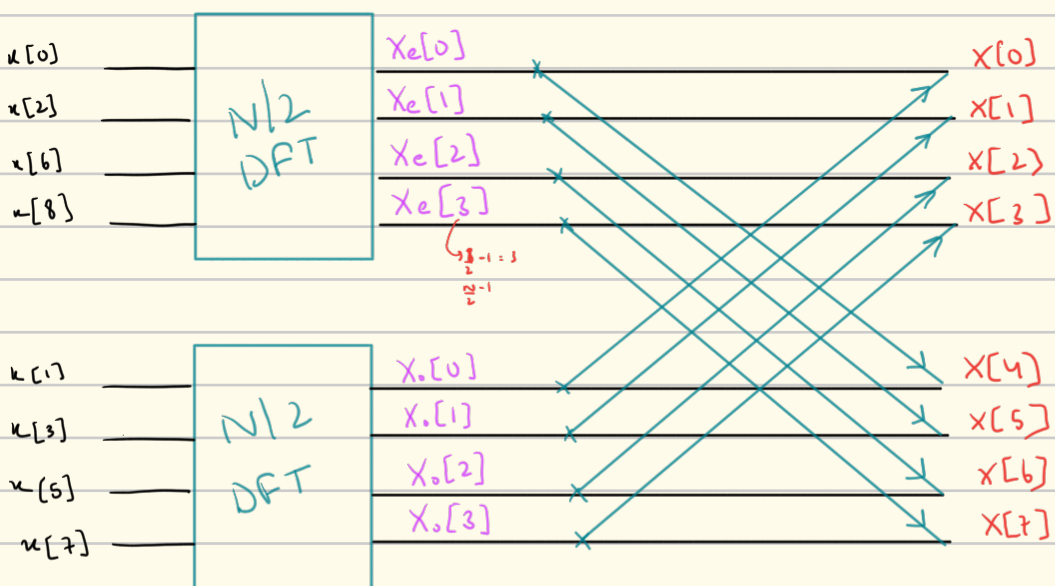
$$= \sum_{r=0}^{\frac{N}{2}-1} x[2r] W_N^{k \cdot 2r} + W_N^k \sum_{r=0}^{\frac{N}{2}-1} x[2r+1] W_N^{2rk}$$

We know $W_N = e^{-j \frac{2\pi}{N}}$, then $W_N^2 = e^{-j \frac{2\pi}{N} \cdot 2} = e^{-j \frac{2\pi}{N/2}}$

$$= \underbrace{\sum_{r=0}^{\frac{N}{2}-1} x[2r] W_{N/2}^{kr}}_{\substack{\text{N/2 DFT of even samples} \\ X_e[k]}} + W_N^k \underbrace{\sum_{r=0}^{\frac{N}{2}-1} x[2r+1] W_{N/2}^{kr}}_{\substack{\text{N/2 DFT of odd samples} \\ X_o[k]}}$$

$X[k] = X_e[k] + X_o[k]$ — sum of 2 N/2 point DFT

example $N=8$



Phase 1

Core Problem with DFT

↳ to find freq content of a specific "bin" k , you loop over every single time sample n , multiply it by a sine/cosine wave and add it up.

→ if you have $N=1024$ samples, you DO $1024 \times 1024 \approx 1,000,000$ multiplications!!

→ $O(N^2)$ time complexity

$$\left\{ \begin{array}{l} \text{butterfly operation: } W_N^k = e^{-j\frac{2\pi k}{N}} \\ \text{to } \downarrow \text{redundant calculation} \end{array} \right\}$$

Phase 2 FFT 'Divide & Conquer' Math (Cooley-Tukey)

• signal $x[n]$
 ↳ even samples → cosine $E[k]$
 ↳ odd samples → sine $O[k]$

↳ twiddle factor

maths says that DFT of full signal = even DFT samples + (odd DFT samples $\times$ twiddle factor)

$$X[k] = E[k] + W_N^k \cdot O[k] \quad (1^{st} \text{ half})$$

• b/c of the periodic nature of sine wave, calculating 2nd half of the freq is practically free

$$X[k+N/2] = E[k] - W_N^k \cdot O[k] \quad (2^{nd} \text{ half})$$

notice that both eq. use the exact same terms ($E[k]$ & $W_N^k \cdot O[k]$)

You compute them once, add them for first half, subtract them for 2nd half.

This 2 ifp or 2 op is called Butterfly operation

→ $O(N \log_2 N)$ power

3 STEPS

1. Bit Sorting: write an alg to swap elements in your array into 'Bit reversed' order
2. Generate twiddle factors: precalculate (or calculate on the fly), the sine & cosine values of W_N^k .
3. Nested Butterfly loops: 3 nested for loops; Outer loop controls STAGE; Middle loop controls 'group of butterflies'; inner loop calculates the actual $E[k] \pm W \cdot O[k]$ maths

STEP 1 Bit Reversal Sorting Alg:

eg: array of $N=8$ samples. $2^3=8 \rightarrow 3$ bits

arr[1] = 001

↓ reverse

arr[1] = 100

so, data sitting at arr[1] needs to be swapped w data sitting @ arr[4]

arr[5] = 011

↓ reverse

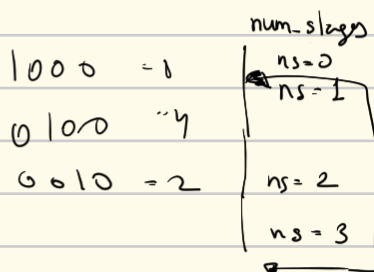
arr[6] = 110

↗ swap data

(while $N < 1$)

$\log_2(N) \rightarrow N \gg 1$

$N=8$



Bit Reversal Alg

- input bin: i
- empty bin: $\text{reversed-}i$
- for ($\dots$; $\dots < \text{num_stages}$; $\dots$)
 - inside loop: 4 steps
 - 1. Extract LSB of i : isolate the bot bit
 $(i \& 1) = i$
Note: if $110 \& 001 = 000$, the 11 is gone from new
sol: create a temp- i , to store old value
 - 2. Make room at $\text{reversed-}i$
slide $\text{reversed-}i$ by one space to the left to make room for extracted LSB.
 $\text{reversed-}i = (\text{reversed-}i \ll 1)$
 - 3. Drop it in
Insert the extracted LSB of i into LSB of $\text{reversed-}i$
 $\text{reversed-}i = (\text{reversed-}i | i)$
 - 4. Shift the i back to normal position
Shift it back right by one again
 $i = (i \gg 1)$

Swapping the Bits:

- Hint 1 guard: swap only if initial index is smaller than reversed index
- Hint 2 problem: we keep to store 1 data before swapping

STEP 2 + 3: Twiddle Factor or Butterfly Math

Scrambling of indexes [step 1]

(N-2)



— Stage 1; dist b/w index = 1: 1 group has 2 people

— Stage 2; dist b/w index = 2: 1 group has 4 people (merged)

— Stage 3; dist b/w index = 4: 1 group has 8 people

→ The Butterfly: One group of 2 is multiply one of them by twiddle factor which is just a point spinning around a sine/cosine circle - and then add/subtract them.

↓
this is what turns Time → Frequency

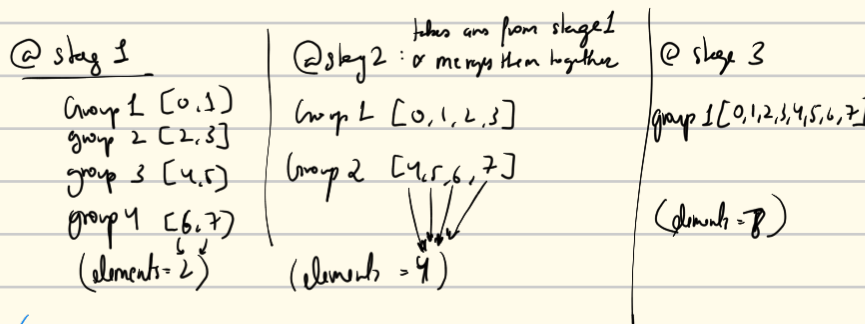
we need 3 nested loops.

next part

Outer loop: Stage loop

- Starts with 1, less than num_stages, i++
- Inside: Calc dist b/w indexes at current stage.

@ stage 1: $N=8$, stage num = 1
 $i \leftarrow \text{stage_num} = 2$
 dist = 1 $\leftarrow \text{stage_num} = 4$
 $i \leftarrow \text{stage_num} = 8$
 (i) (elements in each group)
 in loop



- int num_elements; // no. of elements in each group @ each stage
- int dist_elements; // dist b/w each element @ group @ end stage

Middle loop: Group loop

purpose is to jump b/w groups.

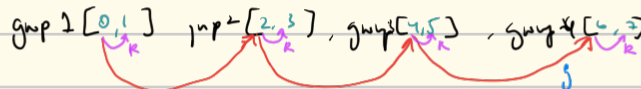
if u are in stage 1: group 1 [0,1], group 2 [2,3], group 3 [4,5], group 4 [6,7]

Main point is jumping b/w groups.

looks @ 0 index then jumps to 2nd, then to 4th then to 6th

Inner loop: Butterfly loop

k must start @ the exact index pointing by j.



TWIDDLE FACTOR:

before the math

two competitors (elements)

we know one is sitting @ k, what about the other (bottom competitor)

Ans) dist b/w both competitors = $\frac{1}{2}$ of group width

- $\frac{0^n}{1 \times 2^n} \rightarrow 0 + \frac{1}{2} \text{ num_elements} = 2^{n-1} \text{ index}$
- $\frac{1^n}{1 \times 2^n} \rightarrow 1 + \frac{1}{2} \times 2^n = 2^{n-1} \text{ index}$

MATH: Angle θ

$$\theta = \frac{-\pi \times \text{Num}}{\text{Denom}}$$

Num: which butterfly uses on (k-j) $\therefore k$ starts @ j this will want 0,1,2...

Denom: half size of the group.

Butterfly Math

↳ in freq. domain $\rightarrow$ REX
 $\rightarrow$ MIX

$$X[k] = E[k] + W_N^k \cdot O[k]$$

↳ twiddle factor $\begin{matrix} \swarrow W^I \\ \searrow W^{-R} \end{matrix}$

maths says that DFT of full signal = even DFT samples + (Odd DFT samples $\times$ twiddle factor)

$$X[k] = E[k] + W_N^k \cdot O[k] \quad (1^{st} \text{ half})$$

• b/c of the periodic nature of sine wave, calculating 2nd half of the freq. is practically free

$$X[k+N/2] = E[k] - W_N^k \cdot O[k] \quad (2^{nd} \text{ half})$$

↳ top compiler just chills, its the bottom compiler that gets multiplied by Twiddle factors

THE FILTER

↳ $\therefore$ data is separated in freq. bins, filtering is insanely fast

↳ we don't need convolution

↳ if you want to delete a freq., you multiply it with 0.